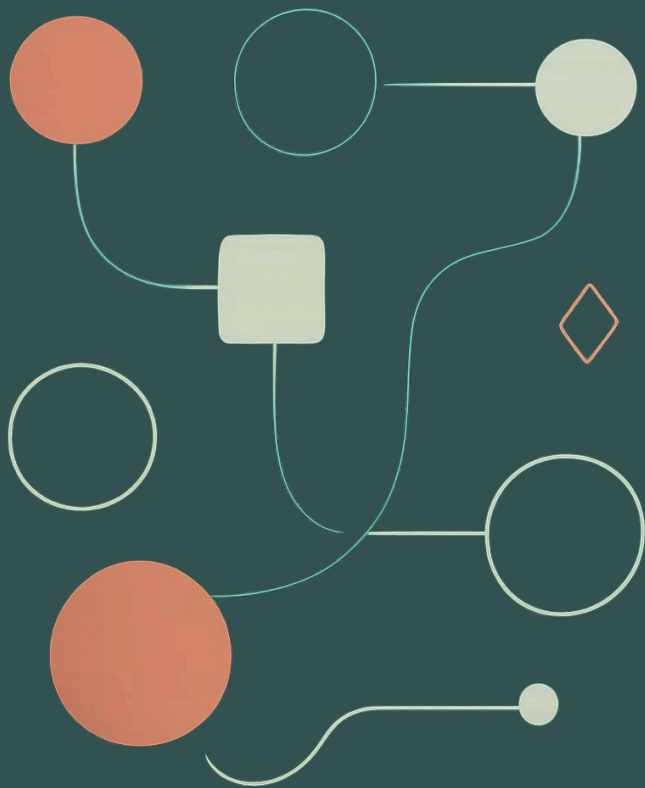




Паттерны проектирования в Java: Chain of Responsibility и Command

Краткий обзор: две фундаментальные схемы для организации ответственности и представления действий в ПО. Эта презентация объясняет идею каждого паттерна, показывает структуру и рабочие примеры на Java, сравнивает их сильные и слабые стороны и подскажет, где применять в реальных проектах.

Что такое паттерны проектирования



Проверенные
решения

Повторяемые
шаблоны для
типовых задач
проектирования.



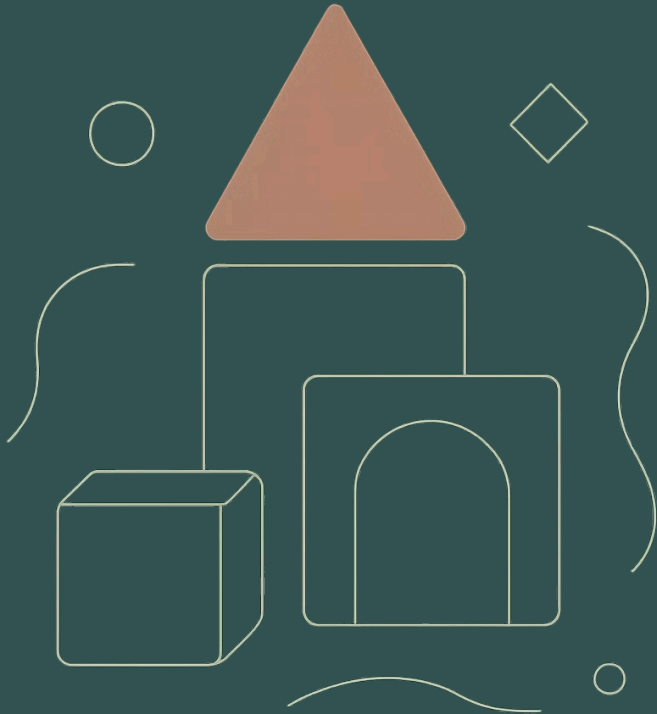
Особенно
важны в Java

Помогают
эффективно
работать в строгой
ООП-модели и
крупных системах.



Упрощают
разработку

Снижают
связанность,
повышают
гибкость и
облегчают
поддержку кода.



Chain of Responsibility — идея и назначение

Идея: передать запрос по цепочке обработчиков до тех пор, пока кто-то не обработает его. Каждый обработчик решает: обработать запрос или передать дальше. Полезен, когда источник запроса не должен знать, кто конкретно его обрабатывает.



Когда использовать
Фильтры, middleware,
логирование, обработка
событий, авторизация по
уровням.



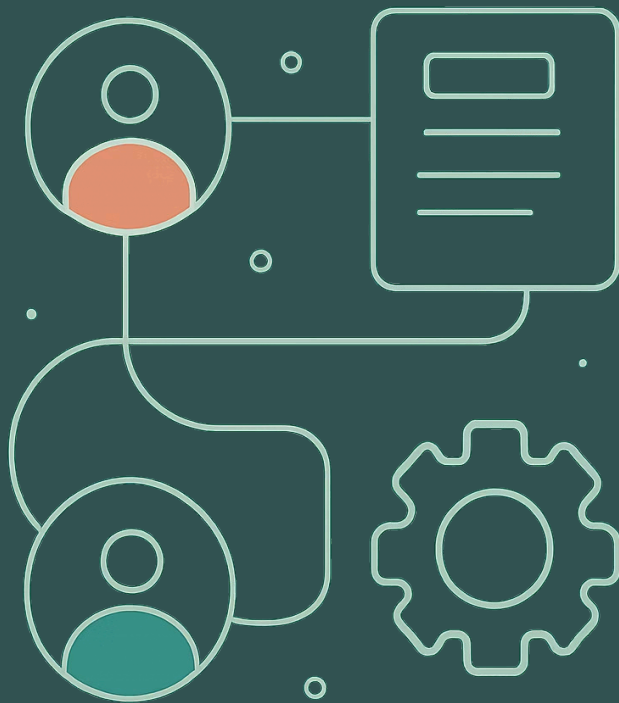
Преимущества

Снижение связанности, гибкое
добавление новых
обработчиков, разделение
ответственности.



Ограничения

Риск необработанного запроса,
сложность отладки длинной
цепочки, возможное влияние на
производительность.



Структура Chain of Responsibility

Основные участники:

- Handler – общий интерфейс/абстрактный класс с методом `handle` и ссылкой `next`.
- ConcreteHandler – конкретные обработчики, реализующие условие обработки.
- Client – формирует запрос и передаёт его в начало цепочки.

Схема работы: Client → Handler1 → Handler2 → Handler3. Каждый элемент может завершить обработку или передать запрос дальше.

Пример Chain of Responsibility на Java

Ниже — упрощённый пример проверки уровня доступа. Handler хранит ссылку на следующий обработчик и вызывает его, если не может обработать запрос.

```
abstract class Handler {
    protected Handler next;
    public void setNext(Handler next){ this.next = next; }
    public abstract void handle(String role);
}
class UserHandler extends Handler {
    public void handle(String role){
        if(role.equals("USER")) System.out.println("Доступ USER подтвержден");
        else if(next!=null) next.handle(role);
    }
}
class AdminHandler extends Handler {
    public void handle(String role){
        if(role.equals("ADMIN")) System.out.println("Доступ ADMIN подтвержден");
        else if(next!=null) next.handle(role);
    }
}
```

Использование: создать обработчики, связать через setNext и передать роль на вход. Результат: соответствующий обработчик выполнит действие.

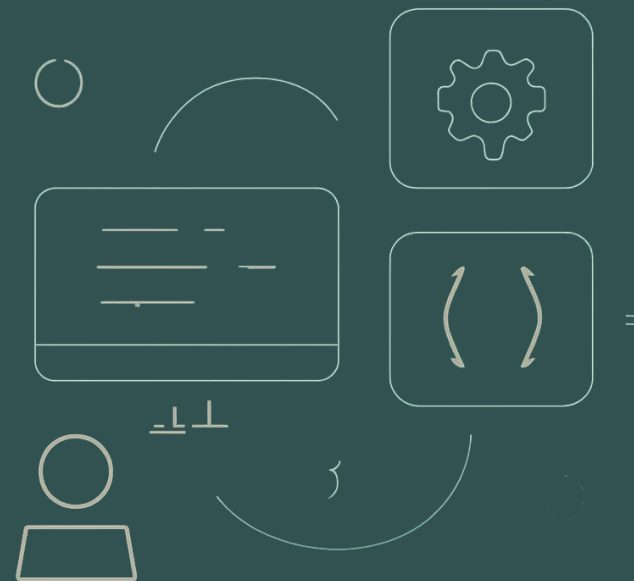
Плюсы и минусы Chain of Responsibility

Плюсы

- ✓ Уменьшает связанность между отправителем и получателями
- ✓ Легко добавлять или переставлять обработчики
- ✓ Разделение ответственности по обработчикам

Минусы

- ✗ Запрос может остаться необработанным без явной обработки по умолчанию
- ✗ Длинная цепочка сложнее дебажить
- ✗ Возможна потеря производительности при глубокой цепочке



DEBUGGING CHAIN
OF HANDLERS

Паттерн Command — идея и назначение

Идея: инкапсулировать запрос в объект-команду. Команда содержит все данные для выполнения операции: ссылку на получателя (Receiver) и параметры. Это позволяет хранить, ставить в очередь, логировать и отменять операции.



Когда использовать

Undo/Redo, очереди задач, отложенное выполнение, макросы, привязка действий к интерфейсу.



Преимущества

Позволяет хранить историю команд, реализовать отмену/повтор, уменьшает связанность Invoker и Receiver.

Пример Command на Java

Классический пример управления лампой: интерфейс Command с методом execute, ConcreteCommand вызывает методы Receiver (Light), Invoker вызывает команду.

```
interface Command { void execute(); }
class Light { void turnOn(){ System.out.println("Свет включен"); } void turnOff(){ System.out.println("Свет выключен"); } }
class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light light){ this.light = light; }
    public void execute(){ light.turnOn(); }
}
class RemoteControl {
    private Command command;
    public void setCommand(Command c){ this.command = c; }
    public void pressButton(){ command.execute(); }
}
```

Использование: создал Light, создал Command, назначил Invoker — при нажатии выполняется действие. Легко расширять новыми командами (вкл/выкл/таймер и т.д.).

Плюсы и минусы Command

Плюсы

- ✓ Разделение Invoker и Receiver
- ✓ Поддержка очередей, логов и отмены операций
- ✓ Удобно для реализации макросов и повторяемых действий

Минусы

- ✗ Увеличение числа классов (много мелких объектов-команд)
- ✗ Сложнее для очень простых задач и маленьких проектов
- ✗ Понадобится дополнительный код для отмены/логирования



Сравнение и рекомендации по применению

Ключевые отличия:

Паттерн	Назначение	Типичные области применения
Chain of Responsibility	Передача запроса по цепочке обработчиков	Фильтры HTTP, middleware, валидация, обработка событий
Command	Представление действия как объекта	Кнопки UI, Undo/Redo, очереди задач, макросы

Практическое правило выбора: если задача — определить кто обработает запрос и разные обработчики могут последовательно решать вопрос → Chain of Responsibility. Если нужно инкапсулировать операцию, хранить её, ставить в очередь или отменять → Command.

Где встречаются в Java

Фреймворки веба (фильтры/интерсепторы), GUI-библиотеки, брокеры задач, паттерны событий и middleware.

Рекомендации

- Используйте Chain для гибкой маршрутизации запросов и валидации.
- Используйте Command для реализации истории действий, отложенного выполнения и undo/redo.
- Комбинируйте: можно передавать команды через цепочку обработчиков.

Заключение: оба паттерна — важные инструменты архитектуры Java-приложений. Понимание их структуры, сильных и слабых сторон помогает проектировать более гибкую и поддерживаемую систему.